Smart Knowledge Base for Agents — Solution Write-up

1. Overview

Two pipelines share a named collection. **Indexing** (`POST /index`) turns an unlabelled folder of PDFs into a navigable library: page-bounded text units, a section tree per document, an LLM-written **card** per document and **"how to tell apart" notes** for look-alike documents, plus a local search index. **Querying** (`POST /query`) runs four steps — **route** → **retrieve** → **grade** → **answer** — of which only route and answer are sequential LLM calls. Every LLM call uses **Claude Haiku 5.5 with thinking disabled** (no hidden reasoning tokens); embeddings and keyword search run locally.

	Value (measured on the training corpus / 40 test questions)
Indexing	≈ 15 min cold for 36 PDFs, 7,091 pages (8 cores); 48 LLM calls ≈ $0.05; re-indexing unchanged files: seconds
Query	23.4 s average · $0.016 · ≈ 130k input + 6k output tokens · 22 LLM calls (≈ 20 of them parallel grading)
Quality (held-out 20)	Answer P 0.37 · Answer R 0.56 · Context P 0.55 · Context R 0.41 (page-based, as the contract scores it)

2. Indexing — `POST /index` (`kb/parse.py`, `kb/index.py`)

2.1 Parse. PyMuPDF 1.26.5 runs in a crash-safe process pool: if a worker dies on a malformed page, the pool is rebuilt and the affected pages are retried one at a time, then without table detection; a page that still fails is kept empty, so one bad page never stops ingestion. Pages with no text layer are OCR'd with Tesseract at 220 dpi (one training PDF is fully scanned). Ruled tables are detected only on pages with vector lines and stored as markdown units. Lines repeated on ≥ 25% of pages (running headers/footers) are removed; text is NFKC-normalised (ligatures `ﬁ` → `fi`) and split thousands separators are re-joined (`60, 000` → `60,000`).

2.2 Structure. Text is accumulated per section and page into **units of 250–2,200 characters (target 1,200) that never cross a page**, so every citation carries an exact page number. Each document gets a **section tree**: from the PDF outline when it has ≥ 5 entries, otherwise from headings detected by dominant font size and weight (wrapped headings merged); OCR'd pages use numbering / capitals rules. Each unit keeps its section path, which retrieval later uses to expand a hit to its section. Training corpus: 24,013 units, 14,580 sections; 0.94 of the gold passages' words are recovered on the correct page.

2.3 Catalogue. One LLM call per document (opening pages, outline, two sampled pages) writes a **document card**: title, short title, entity, issuer, document type, period / fiscal year, date, jurisdiction, status (final / proposed / …), topics, summary. Document identity comes from content, never from the file name. **Look-alike pairs** are found by MinHash over word 5-grams (LSH threshold 0.15) or by the same entity and document type; for each pair an LLM writes a difference note stored on both cards. Real example (12 pairs in the training corpus): "*A is the FY 2024-25 annual report while B is the FY 2025-26 annual report for the same entity. Use A for questions about fiscal year 2024-25 and B for 2025-26.*"

2.4 Index. Every unit is prefixed with a contextual header `[short title | section path | page]`, so year and entity travel with the text into both indexes: **MiniLM** embeddings (local ONNX, no per-query API cost) and **BM25**. Parse output is cached by file hash + parser and library version, embeddings by text hash. A collection is built in a temporary folder and swapped in only when complete; if more than half of the card calls fail, `/index` returns an error and the previous collection is untouched.

3. Querying — `POST /query` (`kb/query.py`)

3.1 route — 1 LLM call, 3.8 s. Input: the question and the whole catalogue (one line per document: card fields + look-alike notes). Output: `question_type` (lookup / analysis / comparison), `facets` (a checklist of what the answer must cover; for a lookup, the question itself), `documents` with reasons, `missing` (documents the question refers to that the collection lacks), 3–6 `search_queries` in the documents' vocabulary, and `confidence`. Prompt rules: match entity, period, version and status exactly using the look-alike notes; prefer a specific document (named reform, proposal, year) over a large compendium. **Near-duplicates are resolved here, at the document level, before any similarity search.** On average 3.8 documents are routed.

3.2 retrieve — no LLM, 0.4 s. (1) Probes = the question + search queries + checklist items. (2) For each probe, dense and BM25 search **restricted to the routed documents**, fused by reciprocal rank; each unit keeps its best score. If routing returned nothing or low confidence, the whole collection is searched as well. (3) Each hit expands to its leaf section when that section is small. (4) Near-identical units (repeated boilerplate) collapse to one. (5) A round-robin across documents fills the candidate budget, so a 1,982-page compendium cannot crowd out a 14-page memo. On average ≈ 150 hits become ≈ 199 candidates.

Question type	**Search hits kept**	**Candidates graded**	**Leaf expansion (max units)**	**Answer length cap**
lookup	30	60	4	150 words
analysis	150	200	10	600 words
comparison	180	240	10	650 words

3.3 grade — parallel LLM calls, 11.5 s. Every candidate is graded for this question: **2** = directly answers a checklist item; **1** = context a grade-2 fact needs (definition, condition, comparison period, caveat); **0** = not needed (topic only, boilerplate, wrong entity / period / version). The test is worded like the evaluator's ("genuinely relevant and useful"); for lookups, grade 2 requires the passage to state the fact. Batches of 10 passages (clipped to 1,800 characters), 24 calls in parallel. If every batch fails, the request returns HTTP 502 rather than a silent "nothing found". On average 38 grade-2 and 54 grade-1 passages; the returned chunks are the grade-2 passages (plus grade-1 when fewer than 15), ≈ 42 per query.

3.4 answer — 1 LLM call, 7.5 s. Written only from grade-2 passages (plus grade-1 when fewer than 8), in document order. Every sentence or bullet ends with `[file pN]`; length is capped near the gold answers' size (table above); a checklist item without evidence gets "The documents do not state …"; missing documents are named. If no passage is grade 2, the system **abstains** ("No information was retrieved…") and returns no chunks.

3.5 Response. `answer`, `chunks` (document, page, section, grade, text), `trace` (one `{step_name, description, input, output}` per step) and `metadata` (tokens, cost, latency, per-step seconds, Langfuse trace id). Every step is a Langfuse span and every LLM call a Langfuse generation with usage and cost.

4. Performance

Answer metrics: the organisers' evaluation notebook code, unmodified (its configured judge, a reasoning model, cannot score long answers within its token budget, so its requests were answered by Claude Haiku 5.5). Context metrics: as the API contract defines them — a chunk is relevant when its document matches a benchmark document and its pages overlap that passage's pages; recall is the share of benchmark (document, page) pairs covered (`scripts/page_metrics.py`, no LLM).

Question set	Answer P	Answer R	Context P (pages)	Context R (pages)	Latency	Cost / query
Development (20)	0.32	0.62	0.53	0.42	24.4 s	$0.017
Held-out (20, never used for tuning)	0.37	0.56	0.55	0.41	22.4 s	$0.016
Answer-judge ceiling (gold answer scored as ours)	≈ 0.95	≈ 0.97				

Latency breakdown (40-question average): route 3.8 s · retrieve 0.4 s · grade 11.5 s · answer 7.5 s. Held-out matches development. Returning grade-1 context passages as well raises page recall to 0.63 at a precision of 0.39 (an earlier configuration), so the chunk-return threshold trades the two context metrics directly.

5. API and operations

Endpoint	Input	Output / errors
`POST /index`	multipart `file` (zip, sub-folders allowed) + `collection_name`	synchronous: 200 `{"status": "success", "collection_name", "documents_indexed"}` once queryable; 400 bad / PDF-less zip; 502 LLM unavailable (previous collection untouched). After 4 min the 200 is started and keep-alive whitespace streamed until the final JSON (tunnels close connections idle for 5 min)
`POST /query`	JSON `collection_name`, `query`	`answer`, `chunks` `[{content, metadata: {file_name, page_numbers}}]`, file names as in the corpus, 1-based pages], four-field `trace`; 404 `{"detail": "unknown collection"}`; 200 with an honest answer and empty chunks when the documents lack the answer; 502 LLM unavailable
`GET /health`	—	`{"status": "ok"}`

Deployment: served from a laptop through ngrok with a static domain (`scripts/tunnel.sh`, keeps the machine awake), as the contract describes; also a Docker image (Python 3.12 + Tesseract, pinned dependencies, embedding model baked in, `/data` volume, `$PORT`) for Railway (`railway.json`) or any Linux server. Stated ingestion time: ~15 min cold for ~7,000 pages on 8 cores (quote 20–25 min with headroom).

6. Design decisions (measured on the same questions)

The current four steps replaced a nine-step version: LLM navigation of the section tree, a gap-fill round, an editor pass and document weighting gave the same scores within noise while costing ~21% more latency and ~14% more money, so they were removed. A claim-bank answer composer raised answer recall (0.65) but cut precision (0.20–0.25), so answers are written from graded passages. Known limits: multi-document regulatory comparisons have the lowest answer precision; tables without ruling lines are parsed as text; only PDFs are indexed; routing sends the whole catalogue in one prompt (comfortable up to a few hundred documents).